```
separate (IDL.SEM_PHASE)¶    --|-----------------------------------------------------------------------------------------¶    --| REP_CLAU¶    --
|------------------------------------------------------------------------------------------¶package body REP_CLAU is¶  use DEF_UTIL;¶  use VIS_UTI
L;¶  use MAKE_NOD;¶  use EXP_TYPE, EXPRESO;¶  use REQ_UTIL;¶  use SET_UTIL;¶  use ATT_WALK;¶  use NOD_WALK;¶¶        -- $$$$ SHOULD NOT BE¬HERE¶
--|-----------------------------------------------------------------------------------------¶        --|¶  procedure REQUIRE_CURRENT_REGION (NAME : TR
EE; DEFSET : in out DEFSET_TYPE; H¬: H_TYPE)¬is¶    TEMP_DEFSET     : DEFSET_TYPE := DEFSET;¶    TEMP_DEFINTERP : DEFINTERP_TYPE;¶    TEMP_DEF        :
TREE;¶    SOURCE_NAME     : TREE;¶    UNIT_DESC       : TREE;¶  begin¶    if IS_EMPTY (TEMP_DEFSET)¬then¶       return;¶    end if;¶¶    DEFSET := EMPTY_
DEFSET;¶    while¬not IS_EMPTY (TEMP_DEFSET) loop¶       POP¬(TEMP_DEFSET, TEMP_DEFINTERP);¶       TEMP_DEF := GET_DEF (TEMP_DEFINTERP);¶¶       if D (XD
_REGION_DEF, TEMP_DEF) = H.REGION_DEF and then not (H.IS_IN_BODY and DB (XD_IS_IN_SPEC, TEMP_DEF)) then¶¬¬¬  -- $$$$¬DON'T KNOW IF DERIVED IS LEGAL $$
$$ ASSUME NOT¶        SOURCE_NAME¬:= D (XD_SOURCE_NAME, TEMP_DEF);¶         if SOURCE_NAME.TY = DN_BLTN_OPERATOR_ID then¶¬null;¶        elsif SOURCE_NA
ME.TY in CLASS_SUBPROG_NAME then¶¬UNIT_DESC¬:= D (SM_UNIT_DESC,¬SOURCE_NAME);¶¬if UNIT_DESC.TY /= DN_RENAMES_UNIT and then UNIT_DESC.TY not in DN_IMPL
ICIT_NOT_EQ .. DN_DERIVED_SUBPROG then¶¬  ADD_TO_DEFSET (DEFSET, TEMP_DEFINTERP);¶¬end if;¶         else¶¬ADD_TO_DEFSET (DEFSET, TEMP_DEFINTERP);¶
   end if;¶     end¬if;¶    end loop;¶¶    if IS_EMPTY (DEFSET) then¶      ERROR (D (LX_SRCPOS, NAME), "MUST BE IN CURRENT REGION");¶    end if;¶  en
d REQUIRE_CURRENT_REGION;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  procedure R
ESOLVE_LENGTH_REP (ATTRIBUTE : TREE;¬EXP : in out TREE; H : H_TYPE) is¶    use PRENAME;¶    ATTRIBUTE_ID : TREE;¶    PREFIX       : TREE := D (AS_NAME
, ATTRIBUTE);¶    PREFIX_TYPE  : TREE := EVAL_TYPE_MARK (PREFIX);¶    TYPESET      : TYPESET_TYPE;¶    STATIC_VALUE : TREE := TREE_VOID;¶  begin¶    A
TTRIBUTE_ID :=¬EVAL_ATTRIBUTE_IDENTIFIER (ATTRIBUTE);¶    PREFIX       :=¬RESOLVE_TYPE_MARK (PREFIX);¶    D (AS_NAME, ATTRIBUTE, PREFIX);¶    D (SM_EX
P_TYPE,¬ATTRIBUTE, TREE_VOID);¶    EVAL_EXP_TYPES (EXP, TYPESET);¶    if ATTRIBUTE_ID¬/= TREE_VOID then¶      case DEFINED_ATTRIBUTES'VAL (DI (XD_POS,
 ATTRIBUTE_ID)) is¶       when SIZE =>¶¬REQUIRE_INTEGER_TYPE (EXP, TYPESET);¶¬REQUIRE_UNIQUE_TYPE¬(EXP, TYPESET);¶¬EXP := RESOLVE_EXP (EXP, TYPESET);
¶¬if not IS_EMPTY (TYPESET) then¶¬  STATIC_VALUE := GET_STATIC_VALUE (EXP);¶¬  if STATIC_VALUE =¬TREE_VOID¬then¶¬    ERROR¬(D (LX_SRCPOS, EXP), "STATI
C EXPRESSION REQUIRED");¶¬  end if;¶¬end if;¶¬if PREFIX_TYPE.TY in CLASS_SCALAR then¶¬  D (CD_IMPL_SIZE, PREFIX_TYPE, STATIC_VALUE);¶¬elsif PREFIX_TYP
E.TY = DN_TASK_SPEC or PREFIX_TYPE.TY in CLASS_UNCONSTRAINED then¶¬  D (SM_SIZE, PREFIX_TYPE, STATIC_VALUE);¶¬end if;¶        when STORAGE_SIZE =>¶¬RE
QUIRE_INTEGER_TYPE (EXP, TYPESET);¶¬REQUIRE_UNIQUE_TYPE¬(EXP, TYPESET);¶¬EXP := RESOLVE_EXP (EXP, TYPESET);¶¬if PREFIX_TYPE.TY /= DN_TASK_SPEC and PRE
FIX_TYPE.TY /= DN_ACCESS then¶¬  ERROR (D (LX_SRCPOS, ATTRIBUTE), "MUST BE ACCESS OR TASK TYPE");¶¬else¶¬  D (SM_STORAGE_SIZE, PREFIX_TYPE, EXP);¶¬end
 if;¶        when SMALL =>¶¬REQUIRE_REAL_TYPE (EXP, TYPESET);¶¬REQUIRE_UNIQUE_TYPE¬(EXP, TYPESET);¶¬EXP := RESOLVE_EXP (EXP, TYPESET);¶¬if not IS_EMPT
Y (TYPESET) then¶¬  STATIC_VALUE := GET_STATIC_VALUE (EXP);¶¬  if STATIC_VALUE =¬TREE_VOID¬then¶¬    ERROR¬(D (LX_SRCPOS, EXP), "STATIC EXPRESSION REQ
UIRED");¶¬  end if;¶¬end if;¶¬if PREFIX_TYPE.TY /= DN_FIXED¬then¶¬  ERROR (D (LX_SRCPOS, ATTRIBUTE), "MUST BE FIXED POINT TYPE");¶¬else¶¬  D (CD_IMPL_
SMALL,¬PREFIX_TYPE, STATIC_VALUE);¶¬  declare¶¬    use UARITH;¶¬    PREFIX_SUBTYPE : TREE := D (SM_TYPE_SPEC, D (SM_DEFN, PREFIX));¶¬    POW_32
  : TREE := U_VALUE ("16#100000000#");¶¬  begin¶¬    if PREFIX_SUBTYPE.TY in DN_PRIVATE .. DN_L_PRIVATE then¶¬      PREFIX_SUBTYPE := D (SM_TYPE_SPEC,
 PREFIX_SUBTYPE);¶¬    elsif¬PREFIX_SUBTYPE.TY =¬DN_INCOMPLETE then¶¬      PREFIX_SUBTYPE := D (XD_FULL_TYPE_SPEC, PREFIX_SUBTYPE);¶¬    end if;¶¬
if PREFIX_SUBTYPE.TY = DN_FIXED then¶¬      D (CD_IMPL_SMALL, PREFIX_SUBTYPE,¬STATIC_VALUE);¶¬    end if;¶¬    D (SM_VALUE, D (AS_EXP1, D (SM_RANGE, P
REFIX_TYPE)), (-POW_32) * STATIC_VALUE);¶¬    D (SM_VALUE, D (AS_EXP2, D (SM_RANGE, PREFIX_TYPE)), (POW_32 - U_VAL (1)) *¬STATIC_VALUE);¶¬  end;¶¬end
if;¶        when others¬=>¶¬ERROR (D (LX_SRCPOS, D (AS_USED_NAME_ID, ATTRIBUTE)), "THIS ATTRIBUTE NOT ALLOWED IN LENGTH CLAUSE");¶¬EXP := RESOLVE_EXP
(EXP, TREE_VOID);¶      end¬case;¶    end if;¶  end RESOLVE_LENGTH_REP;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||¶      --|¶  procedure RESOLVE_ENUM_REP (SIMPLE_NAME : in out TREE; EXP : TREE; H : H_TYPE) is¶    ENUM_TYPE¬ : TREE := EVAL_TYP
E_MARK (SIMPLE_NAME);¶    DUMMY¬¬ : TREE;¶    ENUM_LIST¬ : SEQ_TYPE;¶    ENUM¬¬ : TREE;¶    ITEM_LIST¬ : SEQ_TYPE;¶    ITEM¬¬ : TREE;¶    CHOICE¬ : TR
EE;¶    CHOICE_VALUE¬ : TREE;¶    CURRENT_VALUE¬ : TREE;¶    PRIOR_VALUE¬ : TREE := TREE_VOID;¶    DUMMY_ARRAY_TYPE : TREE;¶  begin¶    SIMPLE_NAME :=
 RESOLVE_TYPE_MARK (SIMPLE_NAME);¶    if ENUM_TYPE.TY¬/= DN_ENUMERATION then¶      DUMMY := RESOLVE_EXP (EXP, TREE_VOID);¶      return;¶    end if;¶¬
    -- (JUST IN CASE IT DOESN'T RESOLVE)¶    D (SM_NORMALIZED_COMP_S, EXP, MAKE_GENERAL_ASSOC_S ((TREE_NIL, TREE_NIL)));¶¶    DUMMY_ARRAY_TYPE := MAK
E_ARRAY (SM_COMP_TYPE => MAKE (DN_UNIVERSAL_INTEGER), SM_INDEX_S => MAKE_INDEX_S (SINGLETON (ENUM_TYPE)));¶    D (SM_BASE_TYPE, DUMMY_ARRAY_TYPE, DUMM
Y_ARRAY_TYPE);¶    DUMMY¬:= RESOLVE_EXP (EXP, DUMMY_ARRAY_TYPE);¶    D (SM_EXP_TYPE,¬EXP, TREE_VOID);¶¶¬      -- CHECK AND INSERT VALUES¶    ENUM_LIST
 := LIST (D (SM_LITERAL_S,¬ENUM_TYPE));¶    ITEM_LIST := LIST (D (SM_NORMALIZED_COMP_S, EXP));¶    if IS_EMPTY (ITEM_LIST) then¶      return;¶    end
if;¶¶    while¬not IS_EMPTY (ENUM_LIST) loop¶      POP¬(ENUM_LIST, ENUM);¶      if IS_EMPTY (ITEM_LIST)¬then¶        ERROR (D (LX_SRCPOS, EXP), "TOO F
EW VALUES");¶        return;¶      end¬if;¶      POP¬(ITEM_LIST, ITEM);¶      if ITEM.TY = DN_NAMED then¶        CHOICE := HEAD (LIST (D (AS_CHOICE_S,
 ITEM)));¶        ITEM   := D¬(AS_EXP, ITEM);¶        if CHOICE.TY = DN_CHOICE_EXP then¶¬CHOICE_VALUE := GET_STATIC_VALUE (D (AS_EXP, CHOICE));¶
  elsif CHOICE.TY = DN_CHOICE_RANGE and then D (AS_DISCRETE_RANGE, CHOICE).TY = DN_RANGE then¶¬CHOICE_VALUE := GET_STATIC_VALUE (D (AS_EXP1, D (AS_DIS
CRETE_RANGE, CHOICE)));¶¬if CHOICE_VALUE /= TREE_VOID and then not UARITH.U_EQUAL (CHOICE_VALUE, D (AS_EXP2, D (AS_DISCRETE_RANGE, CHOICE))) then¶¬  E
RROR (D (LX_SRCPOS, CHOICE), "DUPLICATES NOT ALLOWED");¶¬  return;¶¬end if;¶        else¶¬CHOICE_VALUE := TREE_VOID;¶        end if;¶        if CHOICE
_VALUE = TREE_VOID then¶¬ERROR (D (LX_SRCPOS, CHOICE),¬"MUST BE STATIC, NOT OTHERS");¶¬return;¶        elsif not UARITH.U_EQUAL (CHOICE_VALUE, D¬(SM_P
OS, ENUM)) then¶¬ERROR (D (LX_SRCPOS, CHOICE),¬"MUST EXACTLY MATCH ENUM LITS");¶¬return;¶        end if;¶      end¬if;¶      CURRENT_VALUE¬:= GET_STAT
IC_VALUE¬(ITEM);¶      if CURRENT_VALUE = TREE_VOID then¶        ERROR (D (LX_SRCPOS, EXP), "STATIC VALUE REQUIRED");¶        return;¶      end¬if;¶¶

      if DI (SM_POS, ENUM) /=¬0 and then UARITH."<=" (CURRENT_VALUE, PRIOR_VALUE) then¶        ERROR (D (LX_SRCPOS, EXP), "VALUES MUST BE IN ORDER");¶
      end¬if;¶      PRIOR_VALUE := CURRENT_VALUE;¶      D (SM_REP, ENUM, CURRENT_VALUE);¶    end loop;¶¶    if not IS_EMPTY¬(ITEM_LIST) then¶      ERRO
R (D (LX_SRCPOS, HEAD (ITEM_LIST)), "TOO MANY VALUES");¶    end if;¶  end RESOLVE_ENUM_REP;¶--|######################################################
##########################################¶--|      PROCEDURE RESOLVE_ADDRESS_REP¶  procedure RESOLVE_ADDRESS_REP (SIMPLE_NAME : in¬out TREE;¬EXP : in
out TREE; H : H_TYPE) is¶    NAME_DEFSET : DEFSET_TYPE;¶    NAME_ID      : TREE := TREE_VOID;¶    TYPESET      : TYPESET_TYPE;¶  begin¶    FIND_DIRECT_V
ISIBILITY (SIMPLE_NAME, NAME_DEFSET);¶    REQUIRE_CURRENT_REGION (SIMPLE_NAME, NAME_DEFSET, H);¶    REQUIRE_UNIQUE_DEF (SIMPLE_NAME, NAME_DEFSET);¶
 NAME_ID := GET_THE_ID (NAME_DEFSET);¶    case NAME_ID.TY¬is¶      when CLASS_OBJECT_NAME =>¶        STASH_DEFSET (SIMPLE_NAME, NAME_DEFSET);¶
SIMPLE_NAME¬:= RESOLVE_EXP (SIMPLE_NAME, GET_BASE_TYPE (NAME_ID));¶      when CLASS_SUBPROG_NAME¬=>¶        SIMPLE_NAME¬:= RESOLVE_NAME (SIMPLE_NAME,¬
NAME_ID);¶      when DN_TYPE_ID =>¶        if D (SM_TYPE_SPEC, NAME_ID).TY¬/= DN_TASK_SPEC then¶¬ERROR (D (LX_SRCPOS, SIMPLE_NAME), "ADDRESS CLAUSE NO
T ALLOWED");¶¬NAME_ID := TREE_VOID;¶        end if;¶      when DN_ENTRY_ID =>¶        SIMPLE_NAME¬:= RESOLVE_NAME (SIMPLE_NAME,¬NAME_ID);¶        if D
 (SM_SPEC, NAME_ID).TY = DN_ENTRY and¬then D (AS_DISCRETE_RANGE, D (SM_SPEC, NAME_ID)) /= TREE_VOID then¶¬ERROR (D (LX_SRCPOS, SIMPLE_NAME), "ADDRESS
FOR ENTRY FAMILY");¶¬NAME_ID := TREE_VOID;¶        end if;¶      when DN_VOID =>¶        null;¶      when others =>¶        ERROR (D (LX_SRCPOS, SIMPL
E_NAME), "ADDRESS CLAUSE NOT ALLOWED");¶        NAME_ID := TREE_VOID;¶    end case;¶¶    if PREDEFINED_ADDRESS = TREE_VOID then¶      ERROR (D (LX_SRC
POS, EXP), "PREDEFINED SYSTEM NOT WITHED");¶    end if;¶    EVAL_EXP_TYPES (EXP, TYPESET);¶    REQUIRE_TYPE (GET_BASE_TYPE (PREDEFINED_ADDRESS), EXP,
TYPESET);¶    EXP := RESOLVE_EXP (EXP, TYPESET);¶¶    case NAME_ID.TY¬is¶      when CLASS_OBJECT_NAME =>¶        D¬(SM_ADDRESS, NAME_ID, EXP);¶
 if D (SM_OBJ_TYPE, NAME_ID).TY = DN_TASK_SPEC then¶¬D (SM_ADDRESS, D (SM_OBJ_TYPE, NAME_ID), EXP);¶        end if;¶      when CLASS_SUBPROG_NAME¬| DN
_ENTRY_ID =>¶        D¬(SM_ADDRESS, NAME_ID, EXP);¶      when CLASS_TYPE_NAME =>¶        D¬(SM_ADDRESS, D (SM_TYPE_SPEC,¬NAME_ID),¬EXP);¶      when ot
hers =>¶        null;¶    end case;¶  end RESOLVE_ADDRESS_REP;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||¶      --|¶  procedure RESOLVE_RECORD_REP (SIMPLE_NAME : in out TREE; ALIGNMENT : TREE; COMP_REP_S :¬TREE; H :¬H_TYPE) is¶    DEFSET
: DEFSET_TYPE;¶    NAME_ID      : TREE;¶    NAME_TYPE   : TREE;¶    RECORD_DEF : TREE;¶    TYPESET      : TYPESET_TYPE;¶    REP_LIST    : SEQ_TYPE := LIST (
COMP_REP_S);¶    COMP_REP    : TREE;¶    REP_NAME    : TREE;¶    REP_EXP     : TREE;¶    REP_RANGE   : TREE;¶    IS_SUBTYPE : Boolean;¶    DEFLIST     : SE
Q_TYPE;¶    DEF¬        : TREE;¶  begin¶    FIND_DIRECT_VISIBILITY (SIMPLE_NAME, DEFSET);¶    REQUIRE_UNIQUE_DEF (SIMPLE_NAME, DEFSET);¶    NAME_ID := GE
T_THE_ID (DEFSET);¶    if NAME_ID.TY not in CLASS_TYPE_NAME or else GET_BASE_TYPE (NAME_ID).TY /= DN_RECORD then¶      NAME_ID := TREE_VOID;¶      ERR
OR (D (LX_SRCPOS, SIMPLE_NAME), "MUST BE RECORD TYPE");¶      RECORD_DEF :=¬TREE_VOID;¶    else¶      NAME_TYPE  :=¬GET_BASE_TYPE (NAME_ID);¶      REC
ORD_DEF :=¬GET_DEF (HEAD (DEFSET));¶    end if;¶    SIMPLE_NAME := MAKE_USED_NAME_ID_FROM_OBJECT (SIMPLE_NAME);¶    D (SM_DEFN, SIMPLE_NAME, NAME_ID);
¶¶    WALK_ITEM_S (D (AS_PRAGMA_S, ALIGNMENT), H);¶    if D (AS_EXP, ALIGNMENT) /= TREE_VOID then¶      EVAL_EXP_TYPES (D (AS_EXP, ALIGNMENT), TYPESET
);¶      REQUIRE_INTEGER_TYPE (D¬(AS_EXP, ALIGNMENT), TYPESET);¶      REQUIRE_UNIQUE_TYPE (D (AS_EXP, ALIGNMENT),¬TYPESET);¶      D (AS_EXP, ALIGNMENT
, RESOLVE_EXP¬(D (AS_EXP, ALIGNMENT), TYPESET));¶      if not IS_EMPTY (TYPESET) and then GET_STATIC_VALUE (D (AS_EXP,¬ALIGNMENT)) = TREE_VOID then¶
      ERROR (D (LX_SRCPOS, ALIGNMENT), "ALIGNMENT NOT STATIC");¶      end¬if;¶    end if;¶¶    while¬not IS_EMPTY (REP_LIST) loop¶      POP¬(REP_LIST,
 COMP_REP);¶      if COMP_REP.TY = DN_COMP_REP then¶        REP_NAME  := D (AS_NAME, COMP_REP);¶        REP_EXP   := D (AS_EXP, COMP_REP);¶        REP
_RANGE := D (AS_RANGE, COMP_REP);¶¶        REP_NAME :=¬MAKE_USED_NAME_ID_FROM_OBJECT¬(REP_NAME);¶        D¬(SM_DEFN,¬REP_NAME,¬TREE_VOID);¶        if
RECORD_DEF /= TREE_VOID then¶¬DEFLIST := LIST (D (LX_SYMREP, REP_NAME));¶¬while not¬IS_EMPTY (DEFLIST) loop¶¬  POP (DEFLIST, DEF);¶¬  if D (XD_REGION_
DEF, DEF) =¬RECORD_DEF then¶¬    D (SM_DEFN, REP_NAME, D (XD_SOURCE_NAME, DEF));¶¬    exit;¶¬  end if;¶¬end loop;¶¬if D (SM_DEFN, REP_NAME) = TREE_VOI
D then¶¬  ERROR (D (LX_SRCPOS, REP_NAME), "NOT A COMPONENT OF RECORD");¶¬end if;¶        end if;¶¶        EVAL_EXP_TYPES (REP_EXP, TYPESET);¶        R
EQUIRE_INTEGER_TYPE (REP_EXP, TYPESET);¶        REQUIRE_UNIQUE_TYPE (REP_EXP, TYPESET);¶        REP_EXP := RESOLVE_EXP (REP_EXP, TYPESET);¶¶        EV
AL_EXP_SUBTYPE_TYPES (REP_RANGE, TYPESET, IS_SUBTYPE);¶        REQUIRE_INTEGER_TYPE (REP_RANGE, TYPESET);¶        REQUIRE_UNIQUE_TYPE (REP_RANGE,¬TYPE
SET);¶        REP_RANGE := RESOLVE_DISCRETE_RANGE (REP_RANGE, GET_THE_TYPE (TYPESET));¶        if REP_RANGE.TY = DN_RANGE and then GET_STATIC_VALUE (D
 (AS_EXP1, REP_RANGE)) /= TREE_VOID and then GET_STATIC_VALUE (D (AS_EXP2, REP_RANGE))¬/= TREE_VOID then¶¬null;¶        else¶¬ERROR (D (LX_SRCPOS, REP
_RANGE), "STATIC RANGE REQUIRED");¶        end if;¶¶        D¬(AS_NAME,¬COMP_REP,¬REP_NAME);¶        D¬(AS_EXP, COMP_REP, REP_EXP);¶        D¬(AS_RANG
E, COMP_REP, REP_RANGE);¶      else -- MUST BE PRAGMA¶        WALK (D (AS_PRAGMA, COMP_REP), H);¶      end¬if;¶    end loop;¶  end RESOLVE_RECORD_REP;
¶¶   --|-------------------------------------------------------------------------------------¶end REP_CLAU;¶
```